

Routing3D 라우팅 경로 개발 문서

플랜트 배관 3D 직교 자동 라우팅 엔진 — 프로세스 · 알고리즘 · 핵심 함수 · 변수 · 경로 결과 분석

단위: 모든 좌표·치수 mm · 기본 셀 50mm(뷰어 기본 100mm) · 작성 기준 2026-06

0. 문서 목적과 범위

본 문서는 Routing3D 가 충돌 없는 배관 경로를 어떻게 산출하는가를 개발자 관점에서 상세히 기술한다.

다룬 범위:

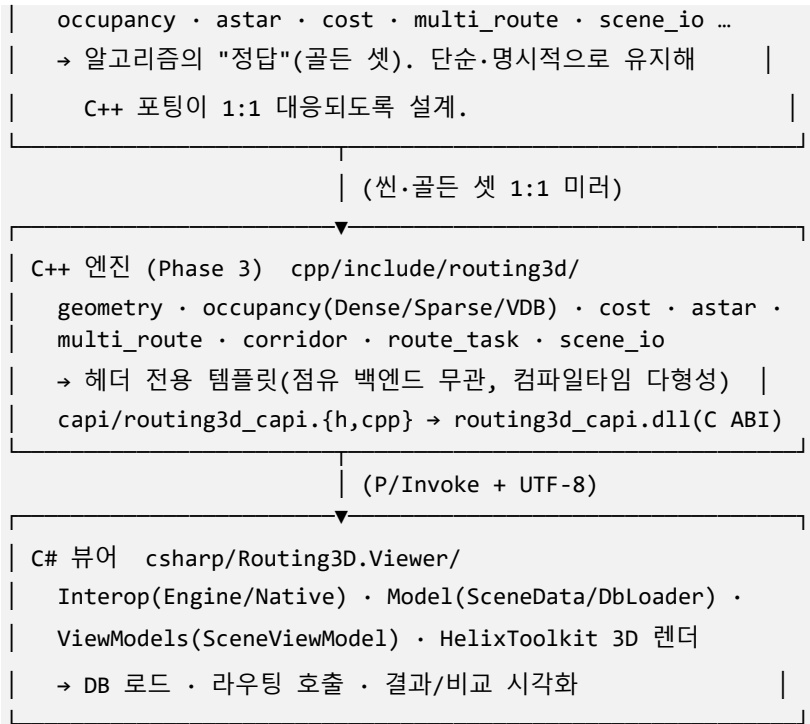
- 라우팅 프로세스(파이프라인): 입력(DB/scene) → 격자·점유맵 → 비용모델 → A* → 다중배관 → 결과
- 각 단계의 알고리즘과 핵심 함수·변수(파일·식별자·수식 수준)
- 엔진의 불변식·결정성 제약
- 경로 결과 분석 지표와 산출식
- C# 뷰어가 더한 라우팅 확장(충돌 대상 확대, 시작 PoC 수직 드롭, 기존설계 비교 분석)

구현은 3-tier(Python 레퍼런스 → C++ 엔진 → C# 뷰어)로 1:1 미러된다. 본문 인용은 C++ 엔진을 기준으로 하고,

Python 레퍼런스(routing3d_py)와의 대응을 병기한다.

1. 아키텍처 개요 (3-tier)

| Python 레퍼런스 (Phase 1) routing3d_py/ |



설계 원칙(엔진): 점유맵 백엔드(Dense/Sparse/OpenVDB)에 무관하게 동일 결과를 보장하기 위해, A*.비용모델은

`template <class Occ>` 로 작성된다. 백엔드는 동일한 질의 인터페이스(`is_blocked/in_bounds/to_cell/to_world/`
`add_box/lin/copy`)만 만족하면 교체 가능하다(불변식 **O1**).

2. 라우팅 파이프라인 (전체 프로세스)

한 프로젝트를 라우팅하는 전체 흐름:

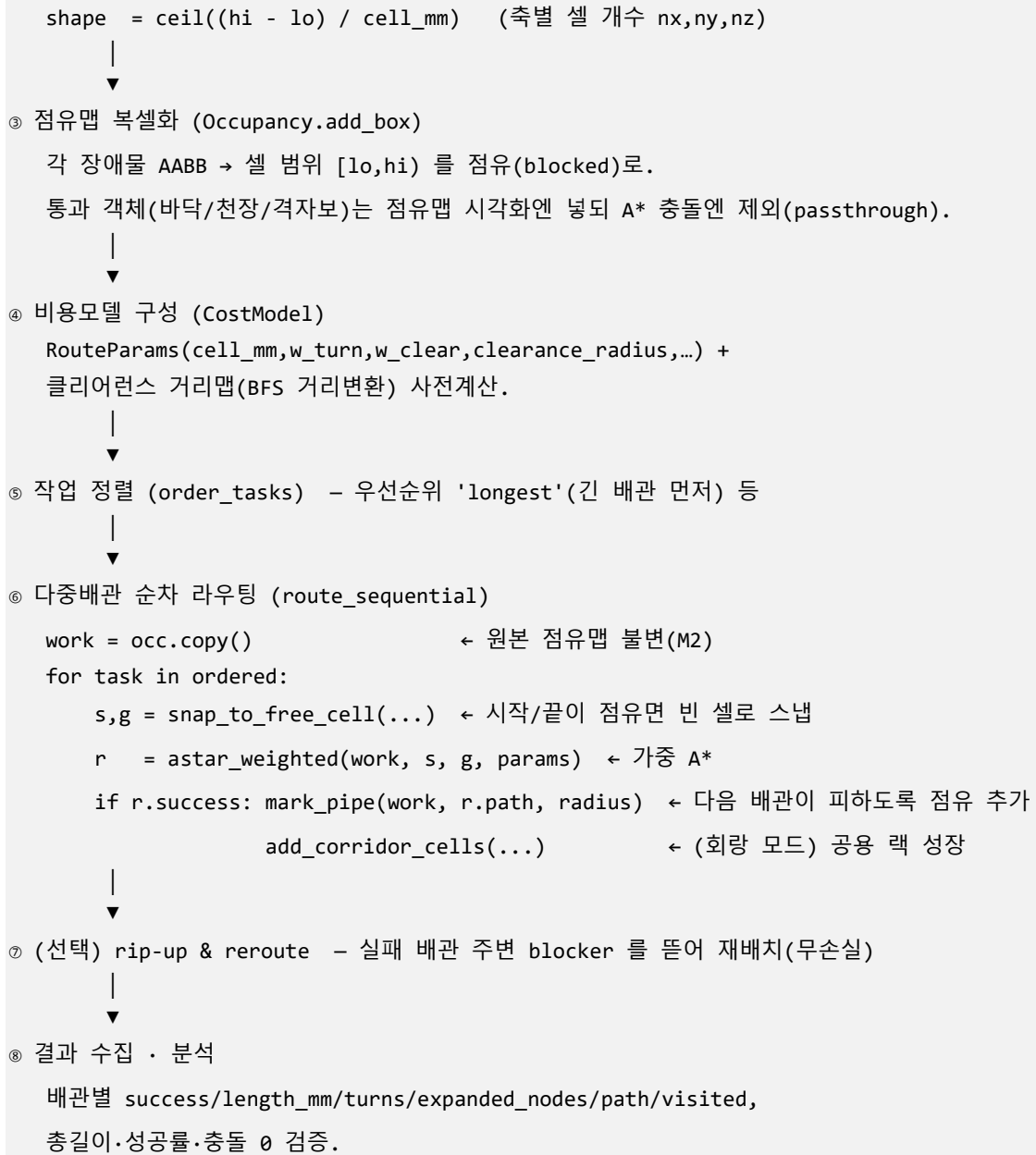
① 입력 수집

- DB(AUTOROUTINGV7): 장애물(TB_BIM_OBSTACLES AABB) + 메인 장비 PoC 페어(TB_BIM_EQUIPMENT.POC_LIST jsonb) → 작업(start→end)
- 또는 scene.txt(격자/장애물/작업 동결 포맷)



② 격자 산출 (ComputeGrid)

origin = 장애물 BBOX 의 lo



대형 격자(8,000m³ 등)는 ⑥ 대신 **계층 corridor 라우팅**(coarse 가이드 → fine tube)으로 즉시 라우팅한다(\$8).

3. 좌표·격자 변환 (geometry.hpp)

모든 좌표는 mm, 격자는 정수 셀 인덱스 `Cell{i,j,k}` 로 표현한다.

3.1 핵심 타입

타입	위치	의미
<code>struct Cell {int i,j,k;}</code>	geometry.hpp:24	정수 격자 좌표(셀 인덱스)
<code>struct Vec3 {double x,y,z;}</code>	geometry.hpp:31	월드 좌표/치수(mm)
<code>struct AABB {Vec3 lo,hi;}</code>	geometry.hpp:39	축 정렬 바운딩 박스(<code>lo<hi</code>)
<code>struct CellRange {Cell lo,hi;}</code>	geometry.hpp:81	셀 범위 [<code>lo,hi</code>](반열림)

3.2 변환 함수 (셀 ↔ 월드)

```
// 셀 중심의 월드 좌표 (geometry.hpp:67)
Vec3 grid_cell_to_world(Cell c, Vec3 origin, double cell_mm)
    = origin + (c + 0.5) * cell_mm

// 월드 좌표를 포함하는 셀 (geometry.hpp:74)
Cell grid_world_to_cell(Vec3 w, Vec3 origin, double cell_mm)
    = floor((w - origin) / cell_mm)
```

- 변환식: `world = origin + (cell + 0.5) * cell_mm`, `cell = floor((world - origin) / cell_mm)`.
- 셀 중심을 쓰는 이유: 경로 셀을 월드로 그릴 때(튜브) 셀 한가운데를 통과하도록.

3.3 AABB → 셀 범위 (복셀화의 기반)

```
// geometry.hpp:88
CellRange grid_box_range(AABB box, Vec3 origin, double cell_mm, Cell shape):
    lo = max( floor((box.lo - origin) / cell_mm), 0 )
    hi = min( ceil((box.hi - origin) / cell_mm), shape ) // 반열림 [lo,hi)
```

- `lo=floor`, `hi=ceil` 로 박스가 걸치는 모든 셀을 포함. 격자 경계 [`0,shape`) 로 클리핑.
- 두께 0/격자보다 얇은 박스도 한 셀 이상 덮도록 호출측에서 팽창해 쓰는 것이 안전(C# 뷰어 `AddBoxObstacle` 의

최소 두께 클램프 참조, §10).

3.4 고정 상수·거리

```
// 6-연결 이웃(면 인접), 고정 순서 - A* 결정성의 핵심 (geometry.hpp:49)
NEIGHBORS_6 = { {+1,0,0},{-1,0,0},{0,+1,0},{0,-1,0},{0,0,+1},{0,0,-1} }

int manhattan(Cell a, Cell b) = |Δi|+|Δj|+|Δk| // (geometry.hpp:54)
```

이웃을 항상 같은 순서로 펼치는 것이 결정성(같은 입력 → 같은 경로)의 한 축이다(불변식 A2/W1).

4. 점유맵 (occupancy.hpp, *_occupancy.cpp)

장애물을 셀 단위 점유(blocked)로 표현. A* 는 점유 셀을 통과하지 못한다.

4.1 공통 질의 인터페이스 (백엔드 무관, 계약 O1)

메서드	의미
<code>bool in_bounds(Cell)</code>	셀이 격자 [0,shape) 내인가
<code>bool is_blocked(Cell)</code>	점유 여부. 격자 밖은 항상 점유 (불변식 G1)
<code>void block_cell(Cell)</code>	셀을 점유로 표시(경계 내만)
<code>int add_box(AABB)</code>	AABB 를 복셀화해 점유 추가, 신규 점유 셀 수 반환
<code>Vec3 to_world(Cell) / Cell to_cell(Vec3)</code>	좌표 변환(geometry 공유)
<code>int lin(Cell) / Cell unlin(int)</code>	셀 ↔ 선형 인덱스(A* g/closed 키)
<code>Occ copy()</code>	깊은 사본(다중배관의 작업용 사본)

```
// 격자 밖 = 점유 (불변식 G1) - 경계 충돌을 자동화
bool is_blocked(Cell c) const {
    if (!in_bounds(c)) return true;
    return /* 백엔드 저장소 조회 */;
}

// 복셀화: AABB 가 덮는 셀 범위를 점유로 (occupancy.cpp:39 / sparse_occupancy.cpp:29)
int add_box(AABB box) {
    CellRange r = grid_box_range(box, origin_, cell_, shape_);
    int newly = 0;
    for (k in [r.lo.k,r.hi.k)) for (j ...) for (i ...)
        if (신규 점유) ++newly;
    return newly;
}
```

4.2 백엔드

백엔드	위치	저장소	용도
DenseOccupancy	occupancy.hpp:24	<code>vector<uint8_t></code> 1B/셀	작은 ROI, 질의 O(1)

SparseOccupancy	occupancy.hpp:67	<code>unordered_set<uint64_t></code> 점유 셀만	초대형 격자(메모리=O(점유))
(VdbOccupancy)	vdb_occupancy.cpp	OpenVDB	8,000m³ 대형

- 선형 인덱스: `lin(Cell) = i + nx·(j + ny·k)`. A* 의 g/closed 배열·맵 키.
- **Sparse 패킹**: `pack(Cell) = (i<<42)|(j<<21)|k` (축당 21 비트, 최대 2^{21} 셀/축).
초대형 격자에선 `lin()` 이

int 범위를 넘을 수 있어 A* 는 경계가 한정된(작은/corridor) 격자에 사용.

Python 대응:

`DenseOccupancyMap/SparseOccupancyMap/BitPackedOccupancyMap(occupancy.py)` — 동일 인터페이스.

5. 비용 모델 (cost.hpp)

A* 가 최소화하는 비용을 정의. **모든 비용항 ≥ 0** (가산 페널티만) → 휴리스틱 admissibility 보존(불변식 C1).

5.1 RouteParams (비용 파라미터)

```
struct RouteParams {           // cost.hpp:25
    double cell_mm = 50.0;      // 셀 크기(mm). geometry 와 일치 필수
    double w_turn  = 500.0;     // 방향 전환(엘보) 페널티(mm) – 부드러운 경로 선호
    double w_clear = 10.0;      // 클리어런스 근접 페널티 계수(mm/셀) – 장애물에서
    // 떨어지게

    int clearance_radius = 2;    // 거리변환 최대 반경(셀)
    int clearance_connectivity = 6; // 6(면) 또는 26(면+모서리+꼭짓점)
    std::map<int,double> w_tier; // z 셀 → 가산(mm), 단(layer) 차별
    double w_corridor = 0.0;     // 회랑 밖 셀 1 개당 가산(mm). >0 이면 배관
    // 번들링(기준설계 유사)

    int corridor_radius = 1;     // 회랑 성장 반경(셀)
    std::vector<int> rack_levels; // 선호 단(z 셀) – 자동 회랑 면제
};
```

파라미터	기본	효과
------	----	----

cell_mm	50	격자 해상도(작을수록 정밀·느림)
w_turn	500	회전 1 회 ≈ 직선 10 칸 비용. 급힘 최소화
w_clear	10	장애물에 1 셀 더 가까울수록 +10mm
clearance_radius	2	이 거리 이상이면 클리어런스 페널티 0
w_corridor	0	0=충돌회피만, >0=공용 랙으로 뚫치게

5.2 클리어런스 거리맵 (BFS 거리변환)

```
// 각 셀에서 가장 가까운 장애물까지 거리(셀, 상한 max_radius) (cost.hpp:53)
vector<int> clearance_map(occ, max_radius, connectivity):
    dist[장애물 셀] = 0; 큐에 push
    다중소스 BFS: dist[이웃] = dist[현재] + 1 (상한 max_radius)
```

- 다중소스 BFS(모든 장애물에서 동시 확장) → O(N). 결과 `clearance_[lin(c)]` 캐시.

5.3 비용 함수

```
// 셀 페널티 (cost.hpp:112) – 모두 가산(≥0)
double cell_penalty(Cell c):
    pen = w_clear * max(0, clearance_radius - clearance[c]) // 클리어런스 근접
        + w_tier[c.k] // 단 분리
        + (회랑 밖이면 w_corridor) // 번들링
    return pen

// 이동 1 회 비용 (cost.hpp:133)
double move_cost(Cell to, Cell* prev_off, Cell move_off):
    c = cell_mm
    if (prev_off && *prev_off != move_off) c += w_turn // 방향 전환
    c += cell_penalty(to)
    return c

// 휴리스틱 (cost.hpp:140) – admissible & consistent
double heuristic(Cell c, Cell goal) = manhattan(c, goal) * cell_mm
```

- **핵심 불변식 C1:** $cell_penalty \geq 0, w_turn \geq 0 \Rightarrow$ 한 칸 이동 $\geq cell_mm \Rightarrow$ 휴리스틱 $h \leq h^*$ (최단 보장).

보너스/감산은 절대 금지(admissibility 깨짐).

Python 대응: `RouteParams/CostModel/clearance_map(cost.py)` — 식 동일.

6. A* 알고리즘 (astar.hpp)

직교 6 방향 A*. 두 변형: 균일 비용 `astar`, 가중 비용 `astar_weighted`.

6.1 결과 구조 (AStarResult, `astar.hpp:29`)

```
struct AStarResult {  
    bool success;           // 경로 발견 여부  
    vector<Cell> path;      // [start..goal], 실패 시 빈 벡터  
    double length_mm;       // (셀 수 - 1) × cell_mm - 기하 길이  
    int turns;              // 방향 전환(엘보) 횟수  
    long long expanded_nodes; // 확장(closed)한 상태 수 - 탐색 효율 지표  
    double cost_mm;         // 페널티 포함 총 비용(가중 A*에서 length 와 다를 수 있음)  
    double elapsed_ms;      // 실행 시간  
    vector<Cell> visited;   // 확장 셀(collect_visited 시) - 방문맵 시각화  
};
```

6.2 균일 비용 A* (`astar`, `astar.hpp:81`)

- 상태 = 셀. 이웃 = NEIGHBORS_6.
- 휴리스틱 $h(c) = \text{manhattan}(c, \text{goal}) \times \text{cell_mm}$, 간선 비용 = `cell_mm`(균일).
- 우선순위 큐 항목 `PQItem{f, counter, cell, dir}`, 비교 (`f` 작은 것, 동률이면 `counter` 작은=먼저 삽입).
- 종료 시 `came` 역추적으로 경로 복원, $\text{length_mm} = (|\text{path}| - 1) \cdot \text{cell_mm}$, $\text{cost_mm} = \text{length_mm}$.

6.3 가중 비용 A* (`astar_weighted`, `astar.hpp:158`)

- 상태 = (셀, 진입방향 `dir`). $\text{dir} \in [-1, 5]$ (-1=시작, 0..5=NEIGHBORS_6 인덱스).
 - 상태 인코딩: $\text{state} = \text{lin} \cdot 7 + (\text{dir} + 1)$. closed 배열 크기 = `size`·7.

- 진입방향을 상태에 넣는 이유: **회전 비용(w_turn)** 을 정확히 매기려면 "어느 방향으로 들어왔는가"가 필요.

- **간선 비용** = `move_cost(to, prev_off, move_off)` = `cell_mm` + (방향전환 시 `w_turn`) + `cell_penalty(to)`.
- 종료 시 `cost_mm` = `g[goal 상태]`(페널티 포함), `length_mm` 은 기하 길이로 별도 계산.
- `collect_visited` 시 셀 단위 중복 제거하여 `visited` 수집.

6.4 결정성 (불변식 A2/W1)

1. **tie-break = (f, counter)**: 같은 f 의 상태는 삽입 순서(counter)로 완전 정렬 → 임의성 제거.
2. **이웃 고정 순서**: 항상 `NEIGHBORS_6` 순서로 펼침.

⇒ 같은 입력은 항상 같은 경로.같은 `expanded_nodes` 를 낸다(재현·회귀 검증 가능). Python `astar/astar_weighted`

(`astar.py`)와 골든 셋에서 `expanded_nodes` 까지 정확히 일치.

7. 다중배관 라우팅 (`multi_route.hpp`)

여러 배관을 서로 충돌 없이 깎다. 순차 + (선택) rip-up.

7.1 작업·결과 구조

```
struct RouteTask { // route_task.hpp:21
    Vec3 start_mm, end_mm;
    optional<string> utility, utility_group, start_name, end_name, end_instance_guid;
    string utility_label(); // "[그룹] 유틸"
};
struct PipeResult { RouteTask task; AStarResult result; int order_index; }; //
multi_route.hpp:38
struct MultiRouteResult<Occ> { // multi_route.hpp:45
    vector<PipeResult> pipes; Occ occupancy; string priority;
    int success_count(); int fail_count(); double total_length_mm(); double
    success_rate();
};
```

7.2 순차 라우팅 (route_sequential, multi_route.hpp:185)

```
MultiRouteResult route_sequential(occ, tasks, params,
    priority="longest", pipe_radius=0, snap_to_free=2,
    max_expansions=-1, collect_visited=false, corridor_radius=1):

    work = occ.copy() // 원본 불변(M2)
    ordered = order_tasks(occ, tasks, priority)
    for task in ordered:
        s = snap_to_free_cell(work, to_cell(task.start_mm), snap_to_free)
        g = snap_to_free_cell(work, to_cell(task.end_mm), snap_to_free)
        r = astar_weighted(work, s, g, params, ...)
        if r.success:
            mark_pipe(work, r.path, pipe_radius) // 점유 추가 → 다음 배관
            add_corridor_cells(corridor, r.path, corridor_radius) // (회랑 모드) 공용
            // 회피
            // 랙
```

보조 함수

함수	위치	역할
<code>order_indices/order_tasks</code>	multi_route.hpp:75	우선순위 정렬(<code>longest</code> =먼 것 먼저, <code>shortest</code> , <code>utility</code> , <code>original</code>) — 안정 정렬
<code>snap_to_free_cell</code>	multi_route.hpp:119	시작/끝이 점유면 반경 내 맨해튼 최근점 빈 셀로(동률은 (di,dj,dk) 사전순)
<code>mark_pipe</code>	multi_route.hpp:141	경로 셀 + 반경 6-이웃을 점유로(다음 배관이 피하도록)
<code>add_corridor_cells</code>	multi_route.hpp:163	경로+반경을 회랑 집합에 추가 → 이후 배관이 길을 싸게 지나 공용 랙으로 뭉침

불변식: **M1** 성공 경로들은 쌍별 셀 비공유(충돌 0) — 깔린 경로를 점유로 추가하므로. **M2** 입력 `occ` 불변(사본 사용).

7.3 Rip-up & Reroute (route_ripup, multi_route.hpp:228)

실패 배관을 구제하는 최적화(무손실·결정적):

- 1) 베이스라인 순차 라우팅(장애물만 기준)
- 2) 라운드 반복(`max_rounds`):

실패 배관 f 마다:

- a) ideal = 장애물만으로 f 경로 탐색(다른 배관 무시)
- b) ideal 과 겹치는 이미 깔린 배관 → blockers
- c) |blockers| > max_ripup 면 스킵
- d) f 재라우팅 + blockers 전부 재라우팅
- e) 모두 성공할 때만 채택(무손실: 성공 수 단조 증가)

- 결정적(std::map 키 순회), 성공 수가 단조 증가 → 유한 라운드 종료.
- 실측: project6 cell=200 에서 multi 77 → ripup 80(+3).

Python 대응:

`route_sequential/route_ripup/order_tasks/_mark_pipe/_snap`(multi_route.py).

8. 계층 Corridor 라우팅 (corridor.hpp) — 대형 격자

8,000m³ 같은 초대형 격자에서 전역 A* 는 메모리 폭발. **coarse 가이드 → fine tube**

2 단계로 푼다.

```
CorridorRoute route_corridor(fine, coarse, start, goal, factor, radius, ...): //
corridor.hpp:139
```

- 1) coarse 격자(셀=factor 배)에서 astar_hashed 로 가이드 경로
- 2) 가이드 경로를 반경 radius(Chebyshev)로 팽창 → corridor 셀 집합(pack20 해시)
- 3) fine 격자에서 corridor 안으로만 제한한 astar_hashed → 최종 경로

- `astar_hashed`(corridor.hpp:51): g/came/closed 를 **해시맵**으로 — 메모리 ∝ 실제 탐색 셀 수(배열 X). 초대형 격자

로컬 배관을 즉시 라우팅.

- `pack20(Cell)=(i<<40)|(j<<20)|k`(축당 20 비트, 8,000m/50mm=160k<2²⁰).
- 결과 `CorridorRoute{fine, coarse_path, coarse_success, corridor_cells}`. 균일 비용(회전/클리어런스 미적용),

작업별 독립(충돌회피 없음).

실측: 8,000m³ 로컬 배관 ~75ms.

9. C ABI 인터롭 (capi/routing3d_capi.h) + C# 통합

C++ 엔진을 외부 의존성 0 의 `routing3d_capi.dll` 로 노출, C# 가 P/Invoke 로 호출.

9.1 POD 구조체

```
typedef struct { double cell_mm, ox,oy,oz; int32_t nx,ny,nz; } R3dGrid;
typedef struct { double cell_mm,w_turn,w_clear,w_corridor;
                int32_t clearance_radius,clearance_connectivity,corridor_radius,
                rack_level_count,rack_levels[8]; } R3dParams;
typedef struct { int32_t success; double length_mm,cost_mm; int32_t turns;
                int64_t expanded_nodes; double elapsed_ms;
                int32_t path_len,visited_len; } R3dResult;
```

9.2 주요 함수 (Level 2 핸들 ABI)

함수	역할
<code>r3d_create/destroy</code>	엔진 핸들 생성/파괴
<code>r3d_set_grid/set_params</code>	격자·비용 파라미터 설정
<code>r3d_add_obstacle / add_passthrough</code>	장애물 / 통과 객체(AABB) 추가
<code>r3d_add_task / set_task_endpoints</code>	작업 추가(→ index) / 종단점 갱신
<code>r3d_route_multi</code>	전체 순차 라우팅(충돌회피)
<code>r3d_route_task</code>	단일 작업(원본 장애물 기준)
<code>r3d_route_ripup</code>	rip-up & reroute
<code>r3d_route_corridor</code>	계층 corridor(Sparse)
<code>r3d_get_result</code>	결과(R3dResult) 조회
<code>r3d_copy_path / copy_visited</code>	경로/방문 셀 복사(int32[3-n])
<code>r3d_copy_blocked / copy_passthrough</code>	점유/통과 셀 복사(점유맵 시각화)
<code>r3d_set_collect_visited</code>	방문 셀 수집 on/off
<code>r3d_load_scene_text / dump_scene_text</code>	scene.txt 로드/덤프

인터롭 안전 규칙: ① 예외는 C ABI 경계를 넘지 않음(try/catch → `R3dStatus`). ② 불투명 핸들 + POD blittable 구조체 +

원시 배열만. ③ cdecl. ④ 문자열 UTF-8(한글). ⑤ 콜리 할당 문자열은 `r3d_free_string`. ⑥ 경로 배열은 2 단계(크기 조회→버퍼).

9.3 C# 래퍼 (Interop/Engine.cs)

- `Engine.SetGrid/SetParams/AddObstacle/AddPassthrough/AddTask/RouteMulti/RouteTask/GetResult/CopyBlocked...`
- 결과
`RouteResult{Success,LengthMm,CostMm,Turns,ExpandedNodes,Path[],Visited[]}`
를 행 캐시(`TaskRowVM.Path/Visited`)에 저장.
- 뷰어는 엔진 상태와 분리해(부분집합 라우팅마다 엔진 재구성) 행 캐시에서 렌더한다.

10. 뷰어 측 라우팅 확장 (C# SceneViewModel.cs)

엔진 위에 뷰어가 더한 실무 규칙. (C++/엔진 무변경, 순수 C# 적재 전략)

10.1 부분집합 라우팅 (BuildEngineForRows)

뷰어는 "모두/그룹별/유틸별/선택 1 개"로 라우팅한다. 매 호출 엔진을 재구성:
격자·파라미터·장애물·작업(해당 행)만 적재

후 `RouteMulti`. 결과를 행 인덱스로 매핑해 캐시.

10.2 충돌 대상 확장 (AddFacilityObstacles) — "충돌확장"

토글(기본 ON)

경로 탐색 충돌 대상을 물리 객체와 시설계 배관까지 확장:

대상	처리
설비(TB_BIM_EQUIPMENT, 메인 장비 포함)	AABB 장애물(<code>AddObstacle</code>)
덕트/레터럴(TB_DUCT_LATERAL)	AABB 장애물. 두께 0 측은 최소 셀로 팽창(<code>AddBoxObstacle</code>)
이미 설계된(라우팅 성공) 다른 배관	경로 셀 폴리라인을 직선 구간별 AABB(~1 셀 두께)로 점유(<code>AddPathObstacle</code>). 자기 자신 제외

- DB 의 기존 설계배관(TB_ROUTE_PATH)은 충돌 대상이 **아님**(비교/참고용).
- 시작/끝 PoC 가 객체 표면에 닿아 막히면 엔진 `snap_to_free_cell`(반경 2)이 인접 빈 셀로 옮김.

10.3 시작 PoC 수직 드롭 (DropStartBelowEquipment)

시작 PoC 가 설비 AABB 내부면(충돌확장 시 설비가 막힘) 수직 하단으로 설비 바닥(MinZ) 한 셀 아래(XY 유지)를 실제

라우팅 시작점으로 삼는다 — 배관이 장비 아래로 빠져나가는 물리적 동작. 옆으로 새는 snap 대신 아래로 내림.

10.4 기존설계 비교 분석 (UpdateComparison/BuildComparison)

선택 배관(시작/끝 PoC)에 대응하는 DB 기존 설계경로를 양방향 최근접으로 매칭(시작↔SOURCE_POS + 끝↔TARGET_POS,

임계 $\max(3 \text{ 셀}, 1500\text{mm})$). 개발 경로(시안)와 기존 경로(주황)를 나란히 그리고 정량 비교:

- **경로 길이**(차이 mm·%)
- **꺾임(엘보) 수** — 수평/수직 분류(수직 = 직전/직후 구간 중 Z 성분 포함)
- **종단점 정합**(시작/끝 mm, 역방향 표기)
- **장애물 간섭/여유** — 경로를 셀/2 간격 샘플 → 비통과 장애물 AABB 내부 점 수(간섭) + 최소 표면거리(여유)

11. 경로 결과 분석 (지표·산출식)

라우팅 품질을 정량 평가하는 지표.

지표	산출식 / 정의	출처
length_mm (기하 길이)	(경로 셀 수 - 1) × cell_mm	astar.hpp / astar.py:205
turns (엘보 수)	인접 셀 이동 방향이 직전과 다를 때마다 +1(count_turns)	astar.hpp:43 / astar.py:111
expanded_nodes	closed 에 들어간 상태 수(가중 A*는 (셀,방향)별) — 탐색 비용	astar.hpp:158 / astar.py:199
visited	확장 셀 목록(셀 단위 중복 제거) — 방문맵 시각화	astar.hpp / astar.py:200
cost_mm	$g[\text{goal}] = \sum \text{move_cost} = \sum (\text{cell_mm} + \text{회전} + \text{페널티})$	astar_weighted / astar.py:312
elapsed_ms	steady_clock 차이	astar.hpp:57

충돌(다중)	성공 경로 셀 집합의 쌍별 교집합(불변식 M1: 0)	multi_route / CollisionFinder.cs
success_rate	성공 배관 / 전체	multi_route.hpp:64
total_length_mm	성공 배관 length 합	multi_route.hpp:58

뷰어 비교 지표(\$10.4): 기존 vs 개발의 길이/꺾임(수평·수직)/종단점 정합/장애물 간섭·여유.

실데이터 교차검증(Python = C++ = C#)

썬	결과
project6 cell=100(장애물 983·작업 208)	194/208 · 3,400,800mm — 3 자 완전 일치
project6 cell=200	multi 77 / ripup 80(+3)
합성 혼잡(9×9 벽+툼 2 개)	seq 1/2 → ripup 2/2

12. 핵심 함수·변수 레퍼런스 (요약)

분류	식별자	위치	핵심
좌표	grid_cell_to_world/grid_world_to_cell	geometry.hpp:67/74	셀↔월드 변환
좌표	grid_box_range	geometry.hpp:88	AABB→셀 범위 [lo,hi)
상수	NEIGHBORS_6	geometry.hpp:49	6 방향 고정 순서(결정성)
점유	is_blocked/add_box/copy	occupancy.*	점유 질의·복셀화·사본
비용	RouteParams	cost.hpp:25	cell_mm,w_turn,w_clear,clearance_radius,w_corridor,rack_levels
비용	clearance_map	cost.hpp:53	거리변환 BFS
비용	cell_penalty/move_cost/heuristic	cost.hpp:112/133/140	가산 페널티·이동비용·맨해튼
A*	astar / astar_weighted	astar.hpp:81/158	균일/가중(상태=셀+방향)
A*	AStarResult	astar.hpp:29	success,path,length_mm,turns,expanded_nodes,cost_mm,visited
다	route_sequential	multi_route.hpp:18	순차 충돌회피

중		5	
다중	order_tasks/snap_to_free_cell/mark_pipe	multi_route.hpp:75/119/141	정렬·스냅·점유
다중	route_ripup	multi_route.hpp:228	rip-up & reroute
회랑	route_corridor/astar_hashed	corridor.hpp:139/51	대형 격자 2 단계
AB	R3dGrid/R3dParams/R3dResult	capi:68/73/82	POD blittable
뷰어	AddFacilityObstacles/DropStarBelowEquipment	SceneViewModel.cs	충돌확장·시작 드롭
뷰어	BuildComparison/FindMatchingExistingPipe	SceneViewModel.cs	기존설계 비교

13. 불변식·결정성 계약 (요약)

ID	불변식	근거
G1	격자 밖 셀 = 점유	is_blocked: !in_bounds → true
O1	백엔드(Dense/Sparse/VDB) 질의 결과 동일	공통 인터페이스 + geometry 공유
A1/C1	비용 가산항 $\geq 0 \rightarrow$ 휴리스틱 admissible & consistent(최적성)	RouteParams ≥ 0 , cell_penalty ≥ 0
A2/W1	동일 입력 $\rightarrow$ 동일 경로	(f, counter) tie-break + NEIGHBORS_6 고정 순서
P1	경로 셀은 비점유, 연속 셀은 6-이웃	A* 확장 규칙
M1	다중배관 성공 경로 쌍별 셀 비공유(충돌 0)	mark_pipe 후 다음 배관 탐색
M2	route_sequential 입력 점유맵 불변	work = occ.copy()
F2~F4	scene.txt 무손실 왕복(repr float, \N vs "")	scene_io

이 계약들이 Python ↔ C++ ↔ C# 3 자에서 동일하게 보존되어 "백엔드·언어 무관 동일 결과"를 보장한다.

문서 끝.